



Understanding Constants and Pointers

Using const qualifiers with pointers is a common source of confusion for most programmers learning C. In this column, we'll discuss the difference between a constant pointer, a pointer to const and a pointer constant.

We have a few declarations here. Can you tell me the difference between them?

```
const char * p1 = 0;
char const * p2 = 0;
char * const p3 = 0;
const char * const p4 = 0;
```

Variables *p1* and *p2* are pointers to *const char*. Here, *const char ** and *char const ** are equivalent. Consider the following statements:

```
p1 = "Hello"; // okay
*p1 = 'h'; // compiler error: assignment of read-only location '*p1'
```

We can change *p1*, but we cannot change the value pointed to by *p1*. That is what we mean by a pointer to *const*. How about the statement given below?

```
char * p = "Hello"; // warning: conversion from string constant to 'char*'
*p = 'h'; // can result in signal 11 (SIGSEGV)
```

Here, *p* is a pointer to *char*, and we initialise it with the string literal "Hello". The compiler warns us that we are trying to convert from a string constant to *char **—why?

A string literal can be stored in a read-only area, and we are not supposed to modify it. So it is of the type *const char **. When we store "Hello", which is of the type *const char ** to *char **, it is not type-safe, and we can end up modifying it. For example **p = 'h'*; tries modifying the literal "Hello". Because of this, I got a signal 11 (segmentation violation) when I tried to run this code. However, it is not necessary that "Hello" be stored in a read-only area, and so it might work fine. However, that leads to 'undefined behaviour', which means, "Don't try it!" Here is another declaration:

```
const char * s = "Hello";
*s = 'h'; // compiler error: assignment of read-only location '*s'
```

This assignment is safe because we cannot modify the value pointed to by *s*—if we try, we get a compiler error. In other words, "Hello" is safe to use through *s*.

Now let's look at *p3*, which is a *const* pointer to *char*. What this means is that we can change it through *p3*, but we cannot change *p3* itself. For example, in the code given below...

```
char ch = 'a';
char * const ptr = &ch;
*ptr = 'b'; // now ch is 'b'
ptr = 0; // compiler error: assignment of read-only variable 'ptr'
```

... *ch* is a character with the initial value 'a'. The variable *ptr*, which is a *const* pointer to character, points to the address of *ch*. When we assign the character 'b' through *ptr*, it modifies the original value in *ch* (i.e., *ch* now has the value 'b'). However, we cannot modify *ptr* itself, since it is a *const* pointer.

With an understanding of *const* pointers and pointers to *const*, it is easy to figure out how to use both of them—and that is as for *p4* below.

```
char ch = 'a';
const char * const p4 = 0;
*p4 = 'b'; // compiler error: assignment of read-only location '*p4'
p4 = &ch; // compiler error: cannot assign to read-only variable '*p4'
```

It would be easy if we had only this 'const pointer' and 'pointer to const', but to add to the confusion, there is also a term called 'pointer constant'! I've seen even experienced programmers get confused between 'pointer to const' and 'pointer constant'. Let's understand the difference now.

```
char array[] = "Hello"; // array is a 'pointer constant'
const char *str = "World"; // str is a 'pointer to constant'
str = array; // okay: we can assign to str; now str is "Hello"
array = str; // compiler error: incompatible types in assignment
```

Array names and function names are examples of 'pointer constants'. We cannot change the addresses themselves, so they are treated as *const* values—in other words, 'pointer constants'. Think of it as similar to constants like integer constants, floating-point constants, etc. On the other hand, in 'pointer to const', we are talking about a pointer that is 'pointing to a const object'. We use a *const* qualifier in the case of a 'pointer to const'. 

About S.G. Ganesh

The author works in Siemens (Corporate Technology), Bengaluru. His latest book is the best-seller *Cracking the C, C++ and Java Interview*, published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com